Used Car Characteristics

An Exploratory and Predictive Data Analysis

Dávid Fodor

Faisal Zayeem Ahmed

Márió Palágyi

Submission date: TBD

Datascience Capstone Project - IMC Krems

Abstract

A brief overview of the project, including problem statement, methodology, key findings, and conclusions.

TODO: FINISH ABSTRACT

1 Introduction

Problem Statement: Describe the problem.

Objectives: List the objectives and tasks required to be completed.

Data: Describe your dataset.

Background Information: Provide context and challenges based on your understanding of the problem.

TODO: FINISH INTRODUCTION

2 Literature Review

Luckily, there is plenty of literature on the topic of used car prices, particularly the prediction seems to be a popular topic. Searching on Google Scholar for "used car price prediction" yields almost a million results.

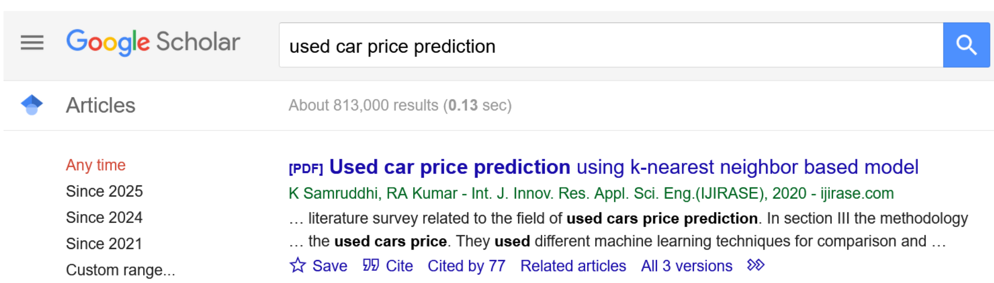


Figure 1: Screenshot of Google Scholar Search. Source: Google Scholar

However, looking for papers that solely try and explore the data and find interesting patterns is lot more difficult. Mostly, the exploration of all the data is done in the context of a prediction model, which is why we also aim to create a usable prediction model as part of our assignment.

Here, as the literature review is a small portion of the report, I will just mention some of the interesting conclusions we have found from others. Firstly, there is seemingly no real consensus on what the best model is for predicting car prices. The the data is usually scraped by the authors [1][2][3], or taken from a source like Kaggle [4][5].

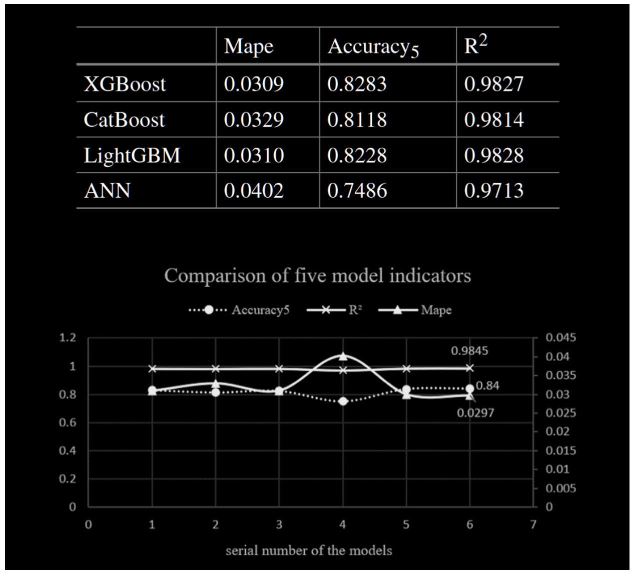
Most of the papers that compared multiple models, found that more complex models, such as **XGBoost** [5], **Random Forest** [6][3] or **Neural Netwroks**[7] worked the best, but the differences were often not so significant, the best models achieved were usually between *85-95%* R^2. There were some surprises too, some have found that rather interestingly other methods worked better for them, such as **SVM** [8], **LightGBM** [9] or **Linear Regression** [2] in case of a smaller dataset.

Some other interesting findings include for example Zheng Y. [4] for whom the most challenging part of price prediction in his case of XGBoost, Random Forest and Linear Regression was of predicting the price of premium cars, the models constantly underestimated them, leading to worse accuracy. The author also suggests taking extra special care when cleaning outliers in price. Chen et al. [10] split their dataset into three distinct groups, once using just cars of the same make and build year, once just a car series and once all data together. They found that random forest worked all round better than linear regression, and that the more records and more features they included, the greater this distance became. This and other papers also suggest that we should not bother with linear regression in cases of larger, more complex datasets. Nandan et al. [5] had a similar experience to us with their large amount of missing values and they also had features reminescent of ours in their Kaggle dataset, they solved the missing values issue with the following method, which they found most optimal and we shall also try to implement in our case:

> "To replenish the missing values in the data, the IterativeImputer technique is employed, with a variety of estimators being developed and their respective MSEs being generated ... the ExtraTreesRegressor estimator is preferable for the imputation strategy in the case of missing value"

Huang et al. [11] had similarly many features to us, leading them to use a forward feature selection which we also plan on implementing due to the large number of features we have. One more interesting thing to note is how they came up with their final prediction, which is a weighted average of their different already well functioning models.

> "...weighted average fusion of XGBoost, CatBoost, LightGBM, and ANN. Its R^2 can reach 0.9845, and the prediction effect is the best."



	Mape	$Accuracy_5$	R^2
XGBoost	0.0309	0.8283	0.9827
CatBoost	0.0329	0.8118	0.9814
LightGBM	0.0310	0.8228	0.9828
ANN	0.0402	0.7486	0.9713

Figure 2: Huang et al. comparative performance of models. Source: [11]

As a conclusion of the literature review, we can safely say that there is no real consensus on what the best model may be for predicting car prices, which may be due to many factors, like the different datasets, different preoprocessing steps and different features. However, we can also see that the more complex models perform better in exploring more nuanced relationships, leading to better predictions as corroborated by Voß S. [6]. Models such as XGBoost and Random Forest seem to be some of the most popular ones, and they are also the ones that we will be expecting to work the best for our case.

3 Methodology

In our methodology section, we will be describing the steps we took to collect our data, process it and what we have found during our exploratory data analysis. We will also be discussing the tools and software we used for this project. Here is a general visualization of the project workflow:

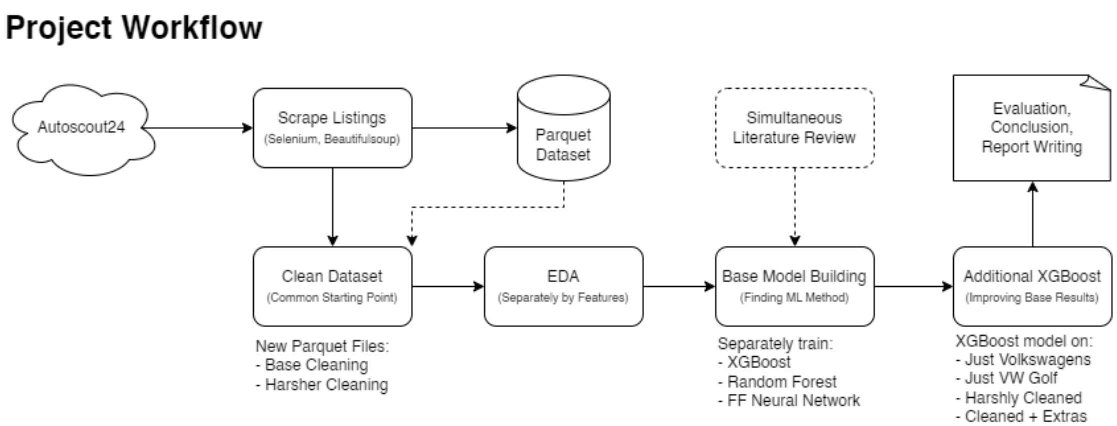


Figure 3: Project Flow. Source: Own work

3.1 Data Collection

In our case the data collection is a rather interesting part of the project, which took quite some time. Unlike other groups, we had to find our own primary datasource, seeking a used car marketplace which we could scrape. After several trials of scraping different websites, we finally ended up with one that was quite lenient with their robots.txt file, and we could scrape it without any issues. The website we used was `https://www.autoscout24.com/`, which is one of the largest used car marketplaces in Europe boasting more than 2 million listings.

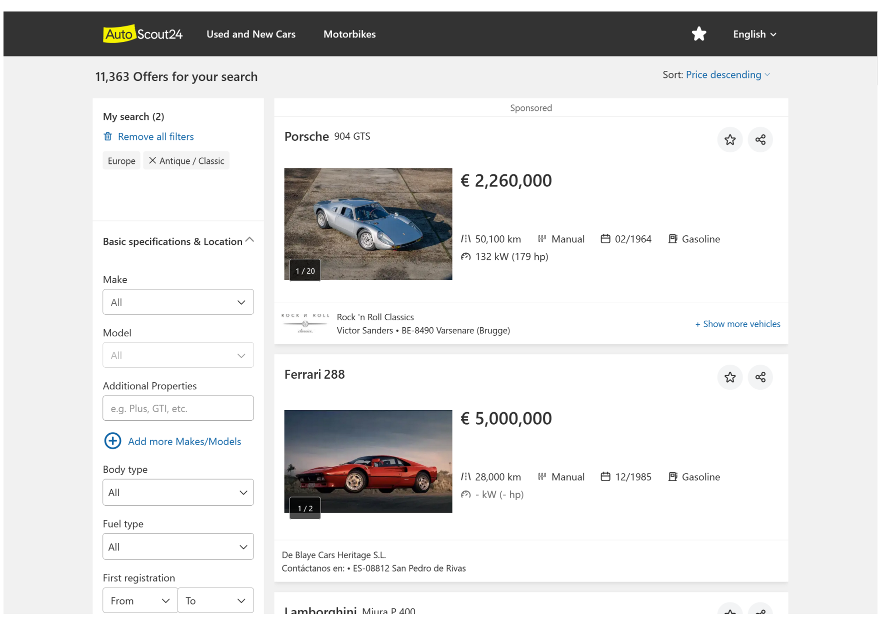


Figure 4: Screenshot of Autoscout24 Homepage. source: Autoscout24

The scraping took place from to , running practically 24/7 on a laptop even though parallelization was used. In the end we have collected *505804* rows of data, which is a lot more than we could have hoped for in, and wrote it to a **parquet** file. The scraping consisted of two main parts, which we will discuss in the following subsections along with the bigger challenges we have faced.

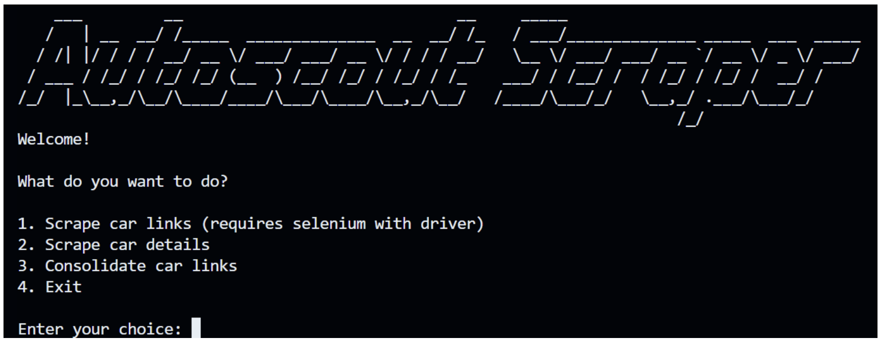


Figure 5: Screenshot of Autoscout Scraper. Source: Own work

3.1.1 Scraping Car Listing

Due to the nature of the website, we needed user interaction for scraping the car listing, so we used **Selenium** to scrape the URLs pointing to specific car listings. This was achieved through a headless Chromium browser. We faced two hurdles in this change which we had to overcome with simulated interactions.

- **Pagination:** The website uses a classic pagination system on page, meaning we needed to be able to navigate through each page.
- **Max 20 pages per search:** The website limits the number of returned pages to *20*, so we had to find a way to methodically filter search parameters to find as many sets of *20* pages as possible.

In theory almost every common feature of cars can be filtered on the page, so with enough patience one may find all 2 million listings. However, we did not have so much time, nor so many cores on our hands, so we tried finding a middle ground, which was the following:

1. Loop through each car make available
2. For each car make, loop through a range of dynamically changing power ranges in kw based on their presumed likelihood e.g. `0-15, 15-20, 20-25, 25-26, 26, 27...`

3. For each power range of the car make, loop through the pages available of the pagination
4. On each pagination page, collect the URLs of the car listings
5. Store the URLs in a JSON file, keyed by the make

This constituted most of our time scraping, as the user interaction needed to be waited upon and there were simply a lot of possible permutations of make, power, pagination possibilities to go through. We of course parallelized and worked on multiple cores, also introducing shortcuts, e.g. if we found a make with less than *500* lisitng we did not need to loop through power ranges at all.

3.1.2 Scraping Car Details

Once we have collected all the car URLs, the scraping was quite straightforward. We have tested the scraping on individual listings first, trying to find out what categories of data we can scrape and how we may achieve that. At this stage, we were lucky, as no user interaction was needed at all, not even for **declining cookies**, as the underlying HTML structure already contained the data we needed. This allowed for us to scrape the data in a much more efficient way, as we could simply use the **requests** package and **BeautifulSoup** to parse the HTML and extract the data we needed. This had much lower overhead and allowed us to scrape more listings simultaneously. First fetching into a list in memory, then converting to a df, to then export into a **parquet** file.

Most of the features scraped are self-explanatory, but there are a few interesting ones from different categories that are notworthy to mention:

- **Metadata:** This includes the **url** of the listing, which is important for us to be able to link back to the original listing.
- **Numerical Features:** These include features such as **price**, **mileage** and **power**. These are seemigly important for our analysis, as they may be the main features that we will be using for our prediction models.
- **Categorical Features:** These include classical features such as **make**, **car_title** and **fuel_type**.
- **Data That Needs Processing:** One good example of this is the **registration_date** feature, which with some simple calculation can be transformed into the **age_months** feature.

- **Additional Text Features:** We also had 4 columns that contained a lot of additional data that was quite sparse so we had to collect them into a single **;** delimitted string, which we later explode into multiple features. Such features are stored in **extras**, **safety_and_security**, **comfort_and_convenience** and **entertainment_and_media**.

Interesting to note that between the two scraping phases, many links seem to have moved, perhaps due to being sold, or delisted so this case also needed to be addressed.

3.2 Preprocessing

We worked separately in numeral Jupyter notebooks, there have been separate notebooks created per each member exploring different aspects of the data. We first started by cleaning the data at the start of each of our own separate notebooks, but later on we decided to do a comprehensive cleaning of the dataset in a separate joint notebook, which can serve as the basis for all of our future work.

The logic can be found in the appropriately named **data_cleaning.ipynb** notebook and it contains not only the cleaning logic, but our reasoning behind each of the cleaning steps. We will now summarize the the steps taken in a nutshell.

First off, we got rid of useless features, this included completely empty, almost empty columns and and simply features that we would not need for any of our analysis, like **manufacturer_colour** for example.

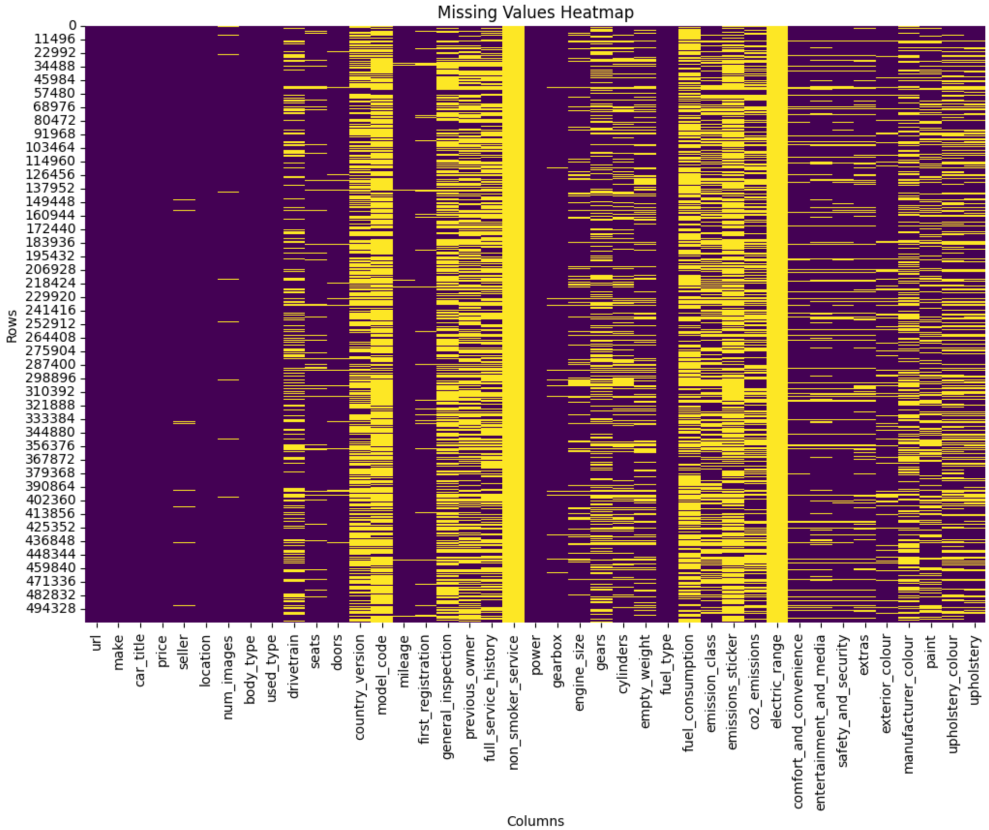


Figure 6: Missing Values Heatmap. Source: Own work

Then, having seen that there are not too many missing values for our main features, **price**, **mileage** and **power**, we decided to drop the rows with missing values, removing *9879* rows, from our 504k large dataset. This seemed like a reasonable choice.

We also had to transform some columns to make them usable for our purposes:

- **Location:** was transformed from the format of `[city, country code]` to just the country code, as we were not interested in the fine grained location of the car.
- **Fuel consumption:** was similarly transformed from `[float] l/100 km (comb.)` string format to just the float value.
- **First registration:** was transformed to **age_months** by subtracting the first registration date from the current date, as we were only interested in the age of the car, not when it was registered.

We found a few imputable / inferable values in our dataset as well:

- **Age:** had seeminlgy *21454* missing values, wehich we could all infer from the column **use_type**, whenever this feature was **New** or **Demonstration**, we filled the missing values with *0*, as these cars were not used at all. This left us with no missing values at all in this column.

- **Fuel Type:** given that the fuel type was electric, we could safely infer that the **co2_emission** was **0** and that the **gearbox** was **automatic**.
- **Full Service History:** only had the value of **Yes** and the lack of this meant that the car did not have a full service history record available, however this was stored as missing value, thus I imputed these with **No**.
- **Previous Owner:** had missing values, which we could infer from the **used_type** feature, whenever this feature was **New** or **Demonstration**, we filled the missing values with *0*, as these cars were not used at all. This left us with no missing values at all in this column.

In the end came the most interesting part, the outlier cleaning. We have inspected 8 of our important numerical features and found that there were some serious outliers in the dataset.

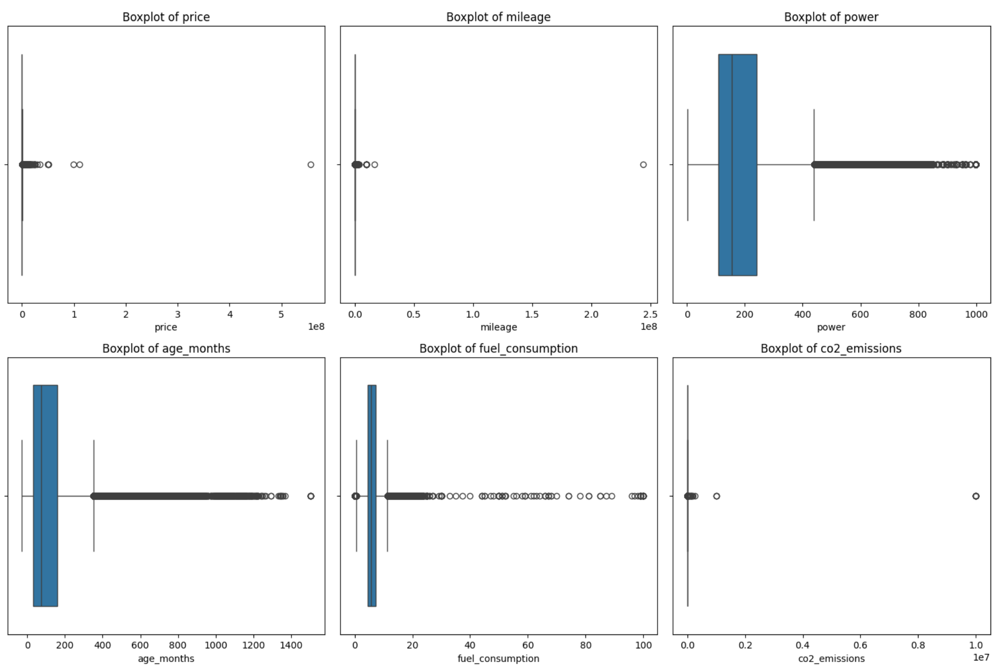


Figure 7: Boxplots of Numerical Features Before Cleaning. Source: Own work

We have used many different strategies to clean the outliers, each based on what felt right for the feature, based on our domain knowledge. Sometimes, we used hard numerical caps, other times used IQR or even 0.05 quantile based capping. As we had so many records to work with, we did not believe winsorization was necessary, so we simply dropped the outliers. One of the more interesting features to clean was the **price** as, this had many outliers on both ends and was arguably our most important feature of all. Here we applied a two step process:

1. We cleaned the outlers with a lenient 2 * IQR based capping, per car make, as this allowed us to keep values that would have been outliers in the general dataset, but were not outliers at all, e.g. Lamborghini.
2. Then we manually inspected both ends of the distribution and set a hard cap of *500* EUR on the lower end and *1000000* EUR on the upper end, cleaning out the most extreme outliers. (This inspection led us to notice interesting things, e.g. towable trailers was listed as a car make, which we removed.)

After removing the outliers for other features, with similar care, but less manual inspection, we were left with a much cleaner dataset, as visible from the following boxplots.

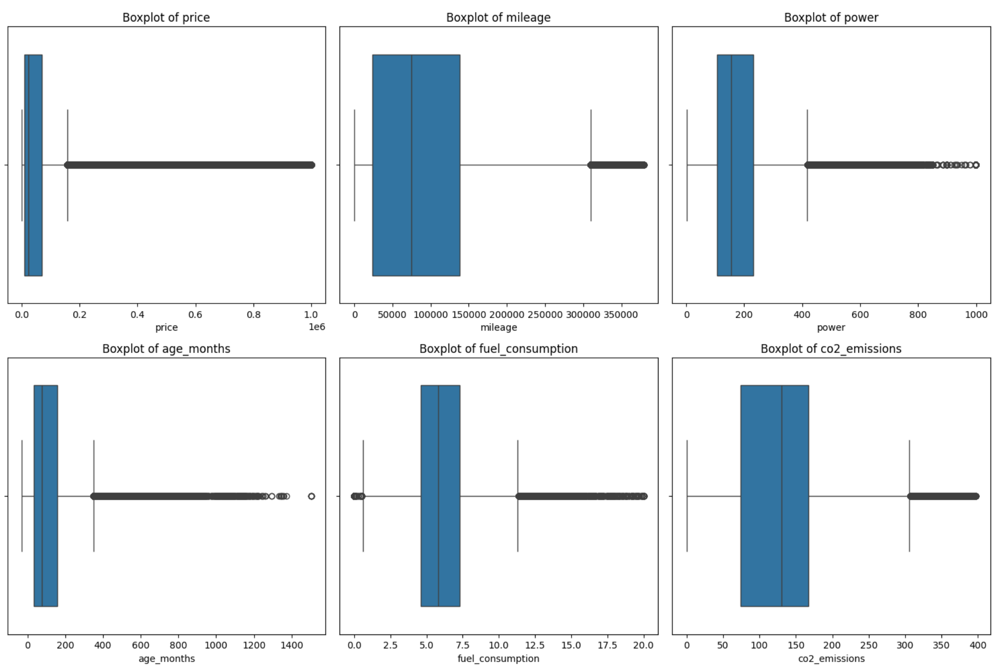


Figure 8: Boxplots of Numerical Features After Cleaning. Source: Own work

The final dataset was then saved to an alternative parquet file, which we will be using for our further analysis. The cleaning here was on purpose not too harsh, such that each subsequent analysis can decide to further clean the data if needed, but we believe we have already found a good balance between keeping the data and cleaning it. In the end this cleaning process removed *37656* rows from our dataset, which is just *7.44%* of the original dataset. This seems reasonable, and we have addressed the the possible downsides of our process as losing all the data for some ultra premium car brands, like Koenigsegg, Paganni and losing some data in case a regular car brand happens to have premium models as well, e.g. Nissan GTR.

We also wanted to create a separate dataset, which is even more cleaned to see if that would yield better results in our prediction models. In the end we used a mixture of IQR and strict numerical capping in conjunction with a by car_title based IQR outlier removal based on our already cleaned dataset from before. This cleaning process can be seen our **harsh_cleaning.ipynb** notebook, at the end of which, we exported the newly cleaned dataset to another parquet file.

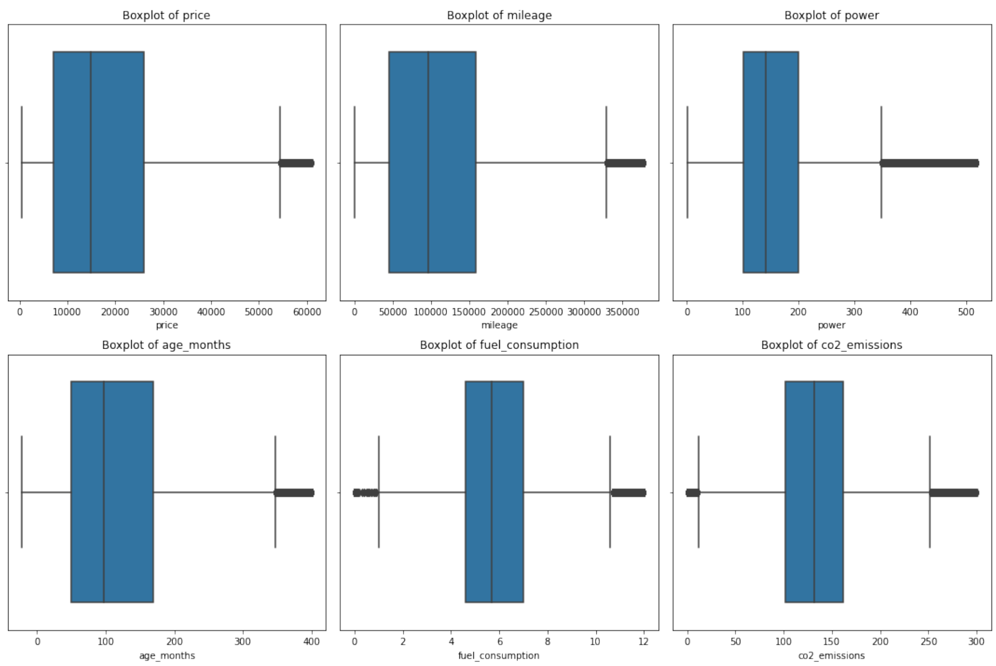


Figure 9: Boxplots of Numerical Features After Harsh Cleaning. Source: Own work

As we can see, the distributions are much nicer and we still have *68.33%* of the original cleaned dataset left, leaving us with *320k* rows. Thus, we can say that this dataset is fairly representative of the average used car market, but it is seriously lacking in premium and luxury cars, as we have capped the prices at around *64k* EUR

3.3 Exploratory Data Analysis

After preprocessing the data, we performed an exploratory data analysis to gain some insights into our freshly harvested dataset. We used various visualization techniques and statistics, but to keep this report concise, we will only present some of the more interesting findings.

One of the first thigns we have checked for was the correlation between numerical features, which we visualized to see if we had mulitcollinearity issues. The matrix showed us there are many features that are highly correlated, as expected, some of the worst offenders were **fuel_consumption VS co2_emissions**, **mileage VS age_months** and **power VS engine_size**. These are all features that we would expect to be collerated, but our main feature we are interested in is the **price**, for which we were happy to see that it had multiple highly correlated features, both positive and negative. Maybe the only really surprisingly useful feature we did not expect to be so highly correlated was the **empyt_weight** of the car, which is strongly positively correlated. In hindsight, this makes sense as heavier cars are usually bigger, more powerful and thus also weigh more, but it is not generally a feature one might think of.

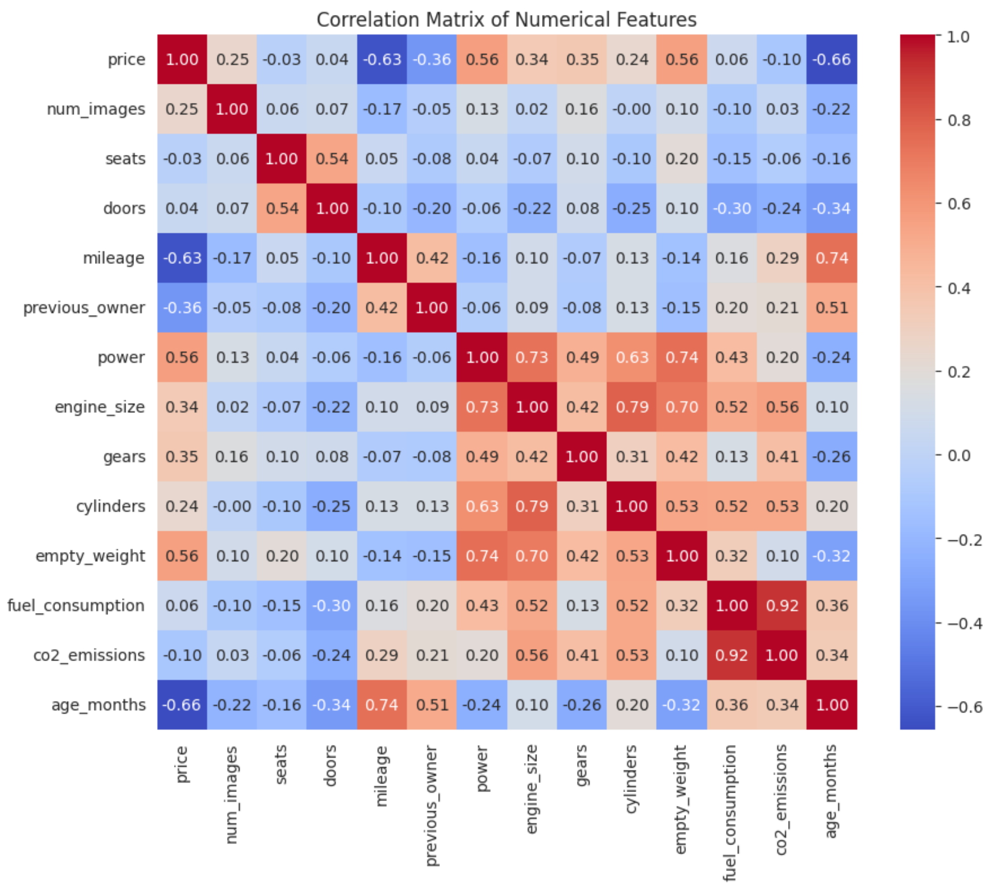


Figure 10: Correlation Matrix of Numerical Features. Source: Own work

TODO: FINISH EDA

3.4 Tools and Software Used

We used a variety of tools and software for this project. The following table summarizes the tools and their respective purposes:

Tool/Software	Purpose
Python	Primary programming language for data scraping, cleaning, and analysis.
Selenium	Web scraping (when user interaction was necessary).
BeautifulSoup	Parsing HTML and gathering data from requests.
Pandas	Data manipulation and analysis.
Numpy	Numerical computing and array operations.
Matplotlib	Data visualization.
Seaborn	Statistical data visualization.
Jupyter Notebook	Interactive development and documentation.
Scikit-learn	Machine learning and model evaluation.
Pytorch	Deep learning models (if applicable).
LaTeX	Typesetting and writing the report.
Zotero	Managing references and citations.
Git	Version control and collaboration.

Table 1: Tools and Software Used

4 Results

Describe experiments and the evaluation protocol. Include tables, graphs, and charts as needed to present your findings and results.

4.1 Prediction Models

As previously mentioned, price prediciton is a very popular topic in the literature, so even though it was not the initial focus of our project, we decided to try and do the best we could with our dataset and make it a prominent part of our report. We have used many different models and techniques for price prediction, and found that it is pretty hard to build an overall good model for all cars as there are many valid outliers in form of premium and luxury cars. That is why we have decided to approach the task from different angles, to see what could be a feasible strategy for predicting the price of used cars. We had the idea of just focusing on one brand, one specific car model and even more harsher cleaning. Here are the main metrics all together in one table:

Method	Mean Absolute Error (MAE)	R-squared (R^2)
XGBoost	29770.0580	0.7949
Random Forest	29822.8709	0.7823
Neural Network	34476.8203	0.7601
XGBoost (Volkswagen)	35747.3092	0.8295
XGBoost (All VW Golfs)	9015.9600	0.8768
XGBoost (Base VW Golfs)	2035.6000	0.8936
XGBoost (Harshly Cleaned)	1806.8305	0.9439
XGBoost (Harshly Cleaned Extras)	1735.6964	0.9496

Table 2: Model Results

Important to note that, the MAE values are not directly comparable here, as the models were trained on different datasets, with different scales of price. For some of the models, you will also find MAPE values in the respective secitons.

4.1.1 XGBoost

XGBoost was shown in some of the papers to be performant, so we decided to try it out as well. We have used the **XGBoost Regressor** from the **xgboost** packages, as we have also seen in this semester's **Machine Learning** course. XGBoost is quite forgiving in terms of preprocessing, so all we had to do was the following:

- **Drop Useless Columns:** We dropped some columns that we deemed useless, such as **url** and some others.
- **Split Data:** We split the data into training and testing sets, using an 80-20 split.
- **Group High Cardinality Features:** We grouped high cardinality categorical features like **car_title** into a more manageable number of categories.
- **Fill Missing Values:** We filled missing categorical values with "Unknown" or similar values.

After all of this, we naturally One-Hot Encoded the categorical features, which left us with quite a few features, *1685* to be exact.

We first did a common quick and easy setup, which already worked quite well, with an R^2 of *0.6965* and a MAE of *47693.6563*. This is already comparable to some of our other models, but of course we wanted to improve on this.

We did some hyperparameter tuning with Randomized Search which gave us a new improved baseline to work with, but was not successful at finding the best parameters, as we could not experiment with so many parameters due to performance and time constraints. After finding these new baseline parameters, we experimented around with more epochs, smaller learning rates, and bigger max depths, trying to find the point where the model would start overfitting to the point where additional complication was not worth it anymore. We think after many rounds of painfully long training and experimentation found a satisfying conclusion to this.

Metric	Train	Test
Average Test Loss (MSE)	948185353.4293	5058389231.1106
Mean Absolute Error (MAE)	14683.8006	29770.0580
R-squared (R^2)	0.9617	0.7949

Table 3: XGBoost Results

We can see that the model managed to learn the training data quite well, with an R^2 of *0.9617*, which in conjunction with our training curves show that we are nearing the level of overfitting, but this is still acceptable and quite an improvement over the baseline performance on the test set too.

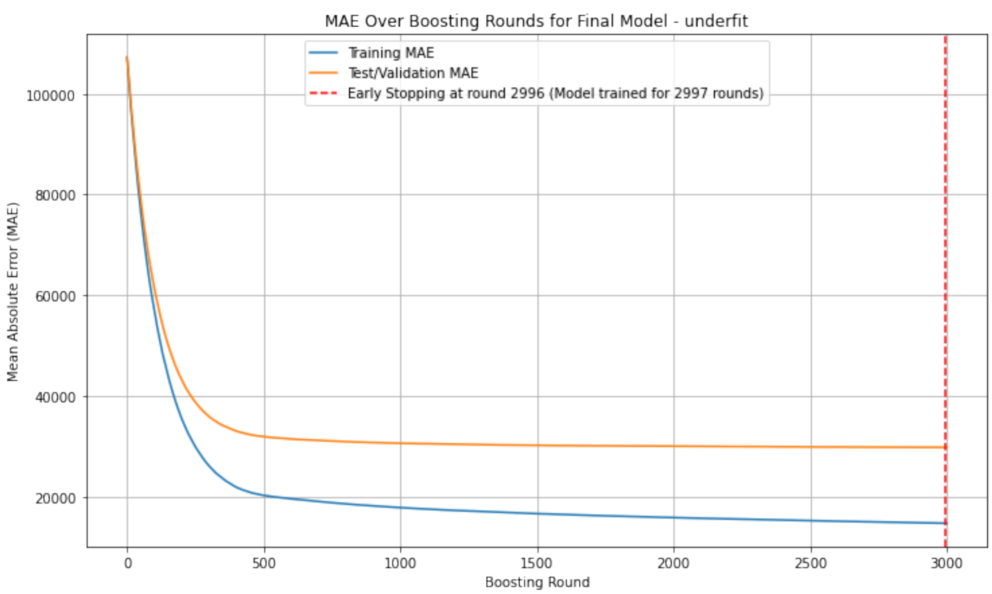


Figure 11: XGBoost Training MAE Cruve. Source: Own work

Plotting the predicted vs actual prices, we can see that the model as in case of many other models too, has a tendency to overestimate some cars under 150k price, which seems serious from the graph, but is not so problematic in context, since most of our cars are under this price range and that is why there are so many points there. The other tendency is unerestimation of luxury and ultra premium cars.

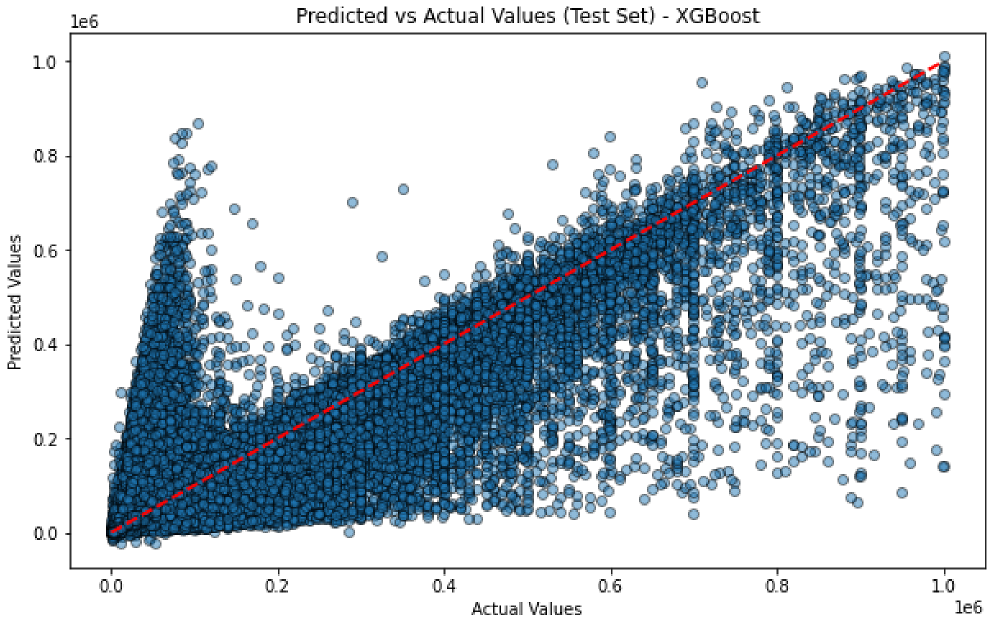


Figure 12: XGBoost Predicted vs Actual. Source: Own work

The histogram of residuals shows us that the vast majority of predictions are quite accurate, it is interesting to compare this to the Neural Network's residual distribution for example, compared to which we can see a much sharper and narrower distribution around the zero point.

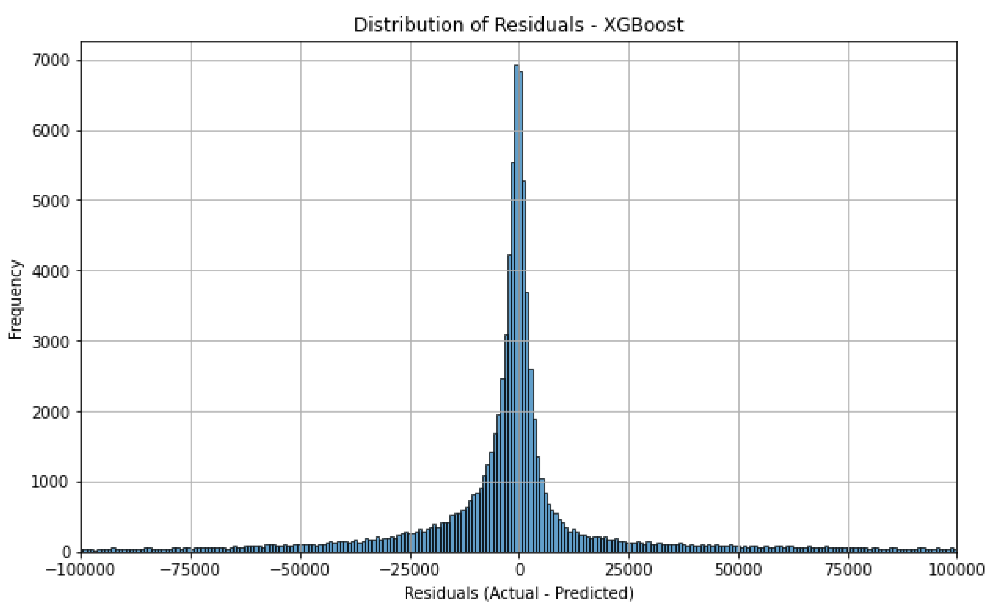


Figure 13: XGBoost Residuals Distribution. Source: Own work

4.1.2 Random Forest

Random Forest is another popular model for this task, and we have used it multiple times too in other courses, so we were quite familiar with it. Setting things up was similar to XGBoost, except for the fact that we had to impute missing values, as these are not handled as well by RF. Here as usual we did grouping of categorical variables, then one-hot encoded them, but what was unique and new here was how we imputed the missing numerical values. Here, we used the by make means of each numerical feature instead of the overall mean, hoping it does a better job at preserving the ditsribtuion of the data. We used the same 80-20 split for training and testing as with the XGBoost and we also got similar results. We as usual first started with a baseline general model, with 100 estimators and full depth trees. Interesingly enough, even after hyperparameter tuning and trying to manually tune the model a bit further we did could not achieve a better result than this baseline. The following table shows the results of the model:

Metric	Train	Test
Root Mean Squared Error (RMSE)	27337.7915	29822.8709
Mean Absolute Error (MAE)	11015.2841	29822.8709
R-squared (R^2)	0.9698	0.7823

Table 4: Random Forest Results

We of course also utilized RF specific features, such as out-of-bag evalutation, which yielded a similar R^2 of 0.7818. One more feature of RF is that we can easily check for feature importance.

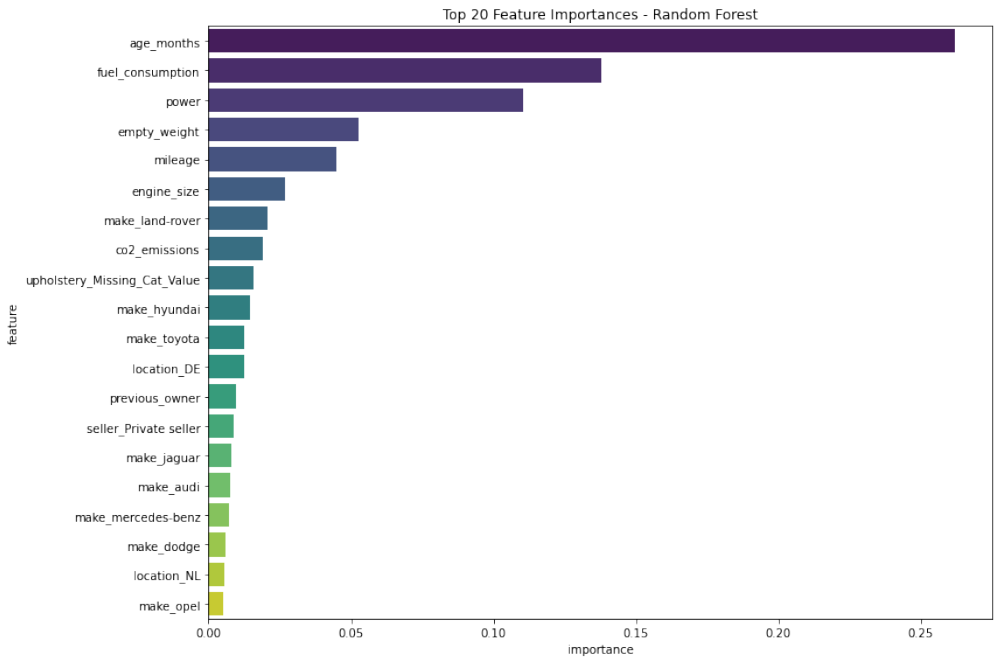


Figure 14: Random Forest Feature Importance. Source: Own work

Firstly, we can see that the numerical features that had high correlation with the price are the most important ones, such as **mileage**, **power** and **age_months**. The categorical features are also quite important, with the individual OHE features of **car_title** and **make** showing up, which is similar to the XGBoost feature importance. Inspecting the predicted vs actual prices, we can see that performance too is identical to the previous model.

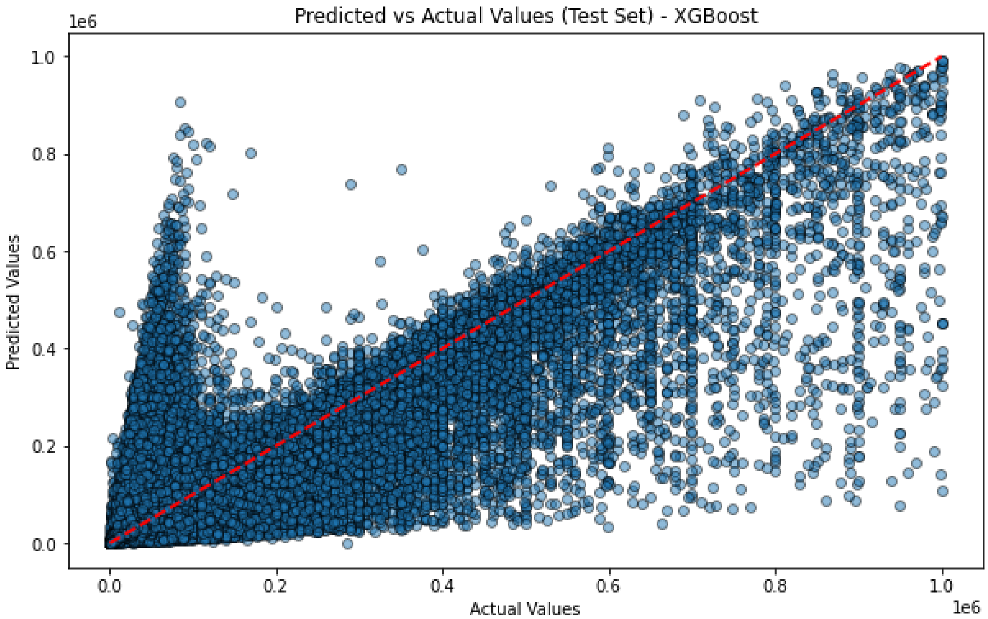


Figure 15: Random Forest Predicted vs Actual Prices. Source: Own work

With the distribution maybe having an even sharper peak around the zero point, leading to many more spot-on predictions.

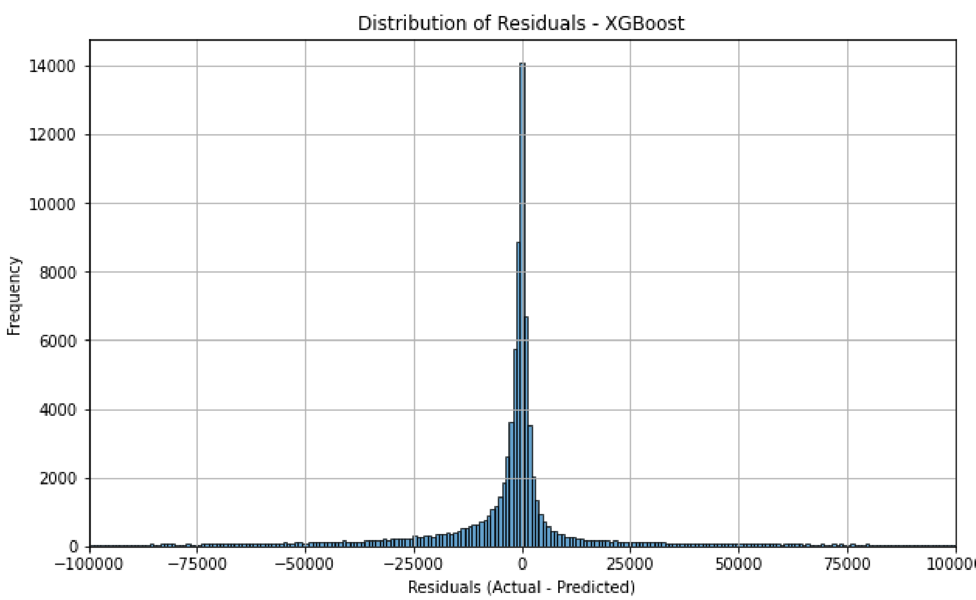


Figure 16: Random Forest Residuals Distribution. Source: Own work

4.1.3 Neural Networks

Neural Networks are not the most popular choice for this task based on the limited literature review we have done, but due to how interesting the topic is we still tried to build a great NN model. Now, the topic of Neural Networks is pretty complex and we most likely do not have the necessary knowledge to properly train one, but we still got an okay result in the end. What made working with Neural Networks challenging was the fact that we had to do even more preprocessing than usual. Not including all steps, here are some of the more interesting steps we had to take:

- **Remove Numerical Features:** We first checked the colinearity of numerical features with price, and decided to remove many columns that did not seem to have any type of effect on the price.
- **Remove Categorical Features:** Then we similarly removed useless categorical features using the Mutual Information method.
- **Grouping and Embedding:** The **car_title** and **make** categorical features with had a lot of unique values, 2400+ and and 200+ respectively, so we first grouped very rare occurences into an "Other" group. Even after this we of course still had a lot of distinct values, so we have used an embedding layer to convert this feature into a dense vector using LabelEncoder instead of just OHE.
- **One-Hot Encoding:** We have used a one-hot encoding for all remaining categorical features, because they did not have too many unique values.
- **Standardization:** We have standardized the numerical features, as they had very different scales.

We also split the data 70-15-15 for training, validation and testing sets. Due, to how NNs work, we had to deal with missing values somehow, so we decided to take two routes and create separate tests for each. Since we have 504k rows, we had the opportunity to simply drop the rows with any missing values, this led to one of our datasets "df_dropped" with still had more than 200k rows. The other dataset "df_imputed" was imputed with the median values and with "Unknown" for the categorical features.

Due to our lack of resources, we had to be smart with how we built and tested our models. Starting out with a simple feedForward NN we tested all of our models both utilizing Embedding and not utilizing it, on both datasets described before. Having run these for 25 epochs, we found that as expected, utilizing Embedding improved the results a lot, but the dropped dataset did not perform better than the imputed one. Following this, we tried hyperparameter tuning our model a bit, which was not so extensive so it only found a few improvements, namely that more layers performed a bit better. We experimented around with 2, 3, 4, 5 and 6 layer models and found that while its loss curve was not the smoothest, the 5 layer model performed the best in case of 20 epochs. As a final step we extended the training to 75 epochs for this model and the 4 layered one to 50 epochs, but the 5 layer one performed better even though it seems to have overfitted a bit and the loss curve is not as smooth as we would have liked it to be.

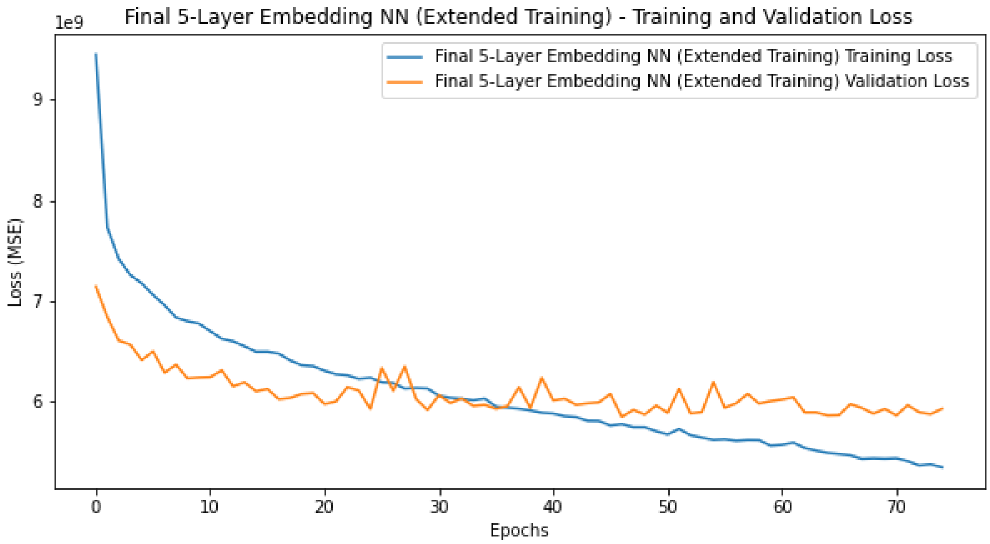


Figure 17: Loss Curve of the 5 Layer Neural Network. Source: Own work

Metric	Value
Average Test Loss (MSE)	5886004206.0474
Mean Absolute Error (MAE)	34476.8203
R-squared (R^2)	0.7601

Table 5: Neural Network Results

This R2 is not the best, but quite decent at *0.7601*. We can see that the Mean Absolute Error is quite big generally, but this is to be expected as we are dealing with a very wide range of prices, from *500* EUR to *1.000.000* EUR. Here are predicted vs actual prices for the test set:

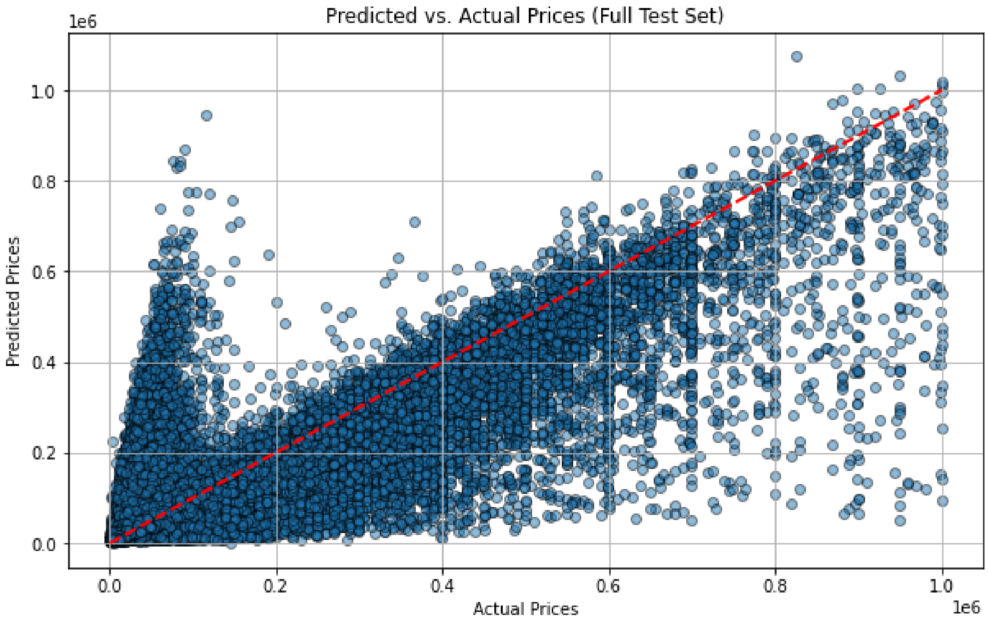


Figure 18: 5 Layer NN - Predicted vs Actual Prices. Source: Own work

As we can see, the model is not perfect, it has a tendency to overestimate some of the cars under 100k and seriously underestimate the premium / luxury cars, which is a common problem in the literature as well.[4]

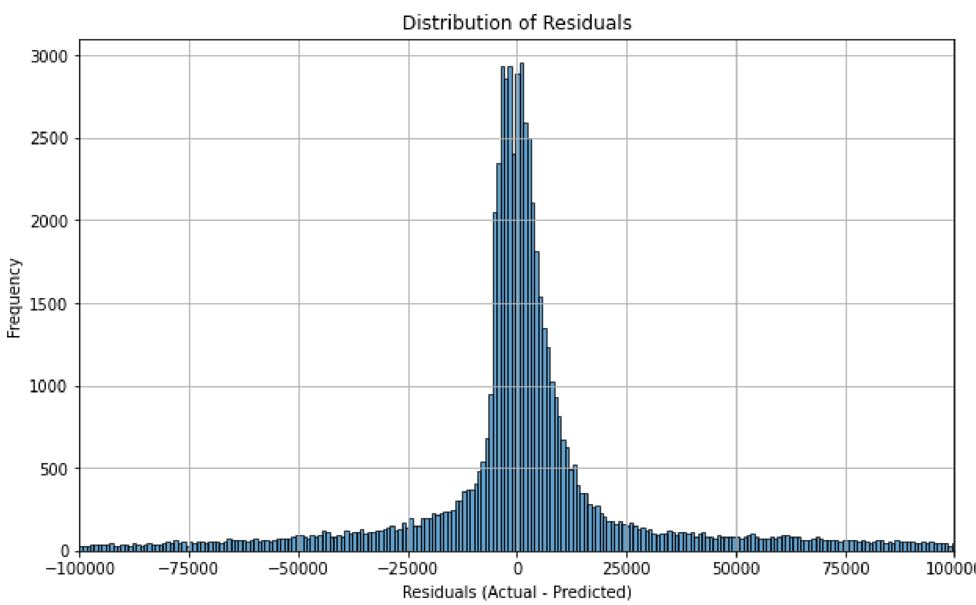


Figure 19: 5 Layer NN - Residuals Histogram. Source: Own work

The resiudals distribution looks okay on both ends, however they are not as sharp as our models.

4.2 Additional Models

Evidently, these models performed okay if we are considering their R^2 scores, but when we lookat at the Mean Absolute Error, the story is completely different, one can not say that a model with a MAE of almost *30000* EUR can be used in practice as an accurate predictor of used car prices. No wonder, there is no such feature on used car marketplace websites, as one may not feasibly train and use just one general model for all cars, given they have such a wide range of prices.

However, it is still intriguing to see how in the literature, many papers have achieved almost near perfect 85-95% R^2 scores, so we decided to try and replicate some of these results with our dataset. Unfortunately, these papers often only share a few details about the dataset they are working with and the cleaning process they had done on it, so we can imagine that they had much stricter cleaning process to optimize the dataset in a way they can boast such high R^2 scores. We tried a few different approaches, such as limiting the dataset to only one brand, limiting the dataset to the most common car via car_title, cleaning the prices more harshly and finally even trying to add some extra features to see if we could improve our best models. For all of these trial due to time constraints we only used XGBoost, as it is very versatile and achieved the best results in our case on the whole dataset.

4.2.1 Limiting to One Brand - VW

The reasoning behind linitng the dataset to just one make is through this not only would we potentially get rid of make as a factor, but supposedly VW is a common brand that does not have so many outliers. . . Or so we have thought. In reality, VW still had many premium cars which could be considered outliers up to the 600k euros range. Unsurprisingly, the results we got were very familiar to what we had with the whole dataset. After some trial and error, hyperparameter tuning, which this time took a lot less time we achieved the following best statistics:

Metric	Train	Test
Mean Absolute Error (MAE)	8490.7918	35747.3092
Mean Squared Error (MSE)	260848128.6913	4956616019.6742
R-squared (R^2)	0.9909	0.8295

Table 6: XGBoost Results on VW Dataset

The model could learn the training data really well and it also led to an increased R^2, but our main metric we wanted to optimize more, MAE has not become any better, on the contrary, it was worse than on the overall set. Another interesting thing to note is, since car make is removed from the set of features, our feature importances look drastically different, with less common categorical features making an appearance.

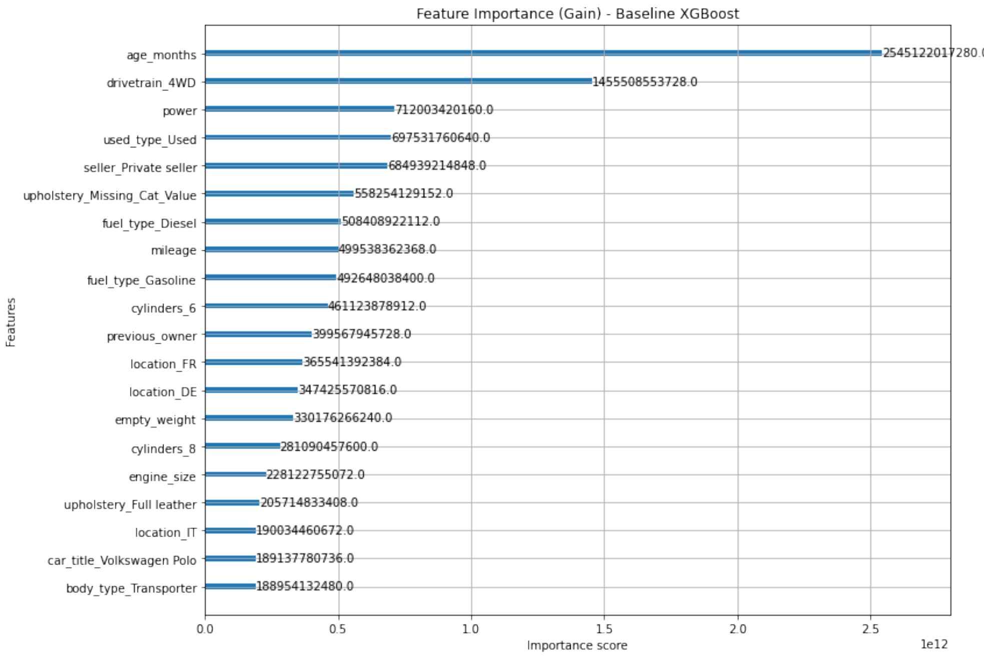


Figure 20: XGBoost Feature Importance on VW Dataset. Source: Own work

Our residuals plots are almost identical to the ones using the whole dataset, except there are less records and it is easier to inspect.

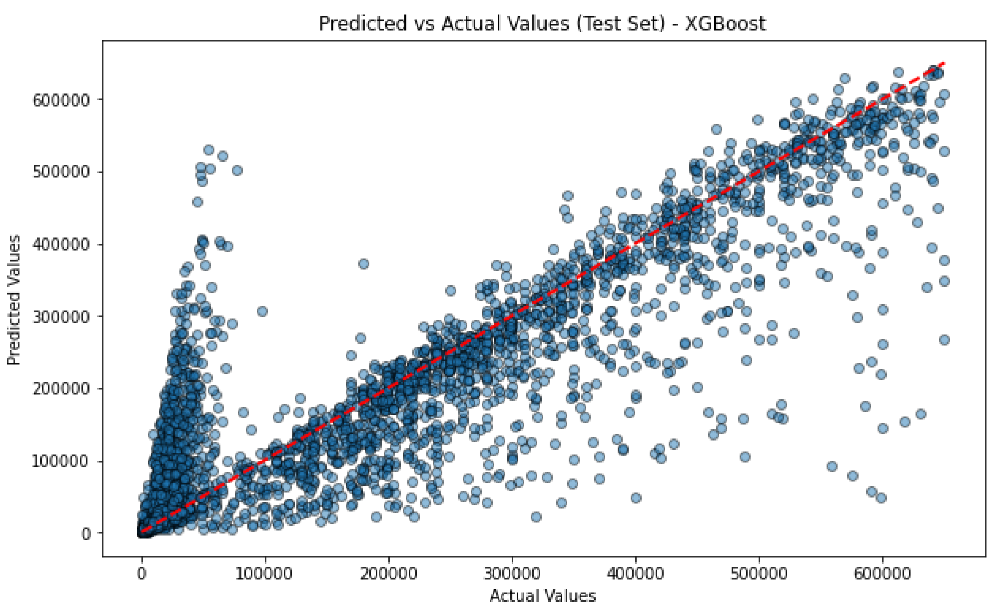


Figure 21: XGBoost Predicted vs Actual Prices on VW Dataset. Source: Own work

4.2.2 Limiting to One Car - VW Golf

We saw that from before, that simply limiting the dataset to one make does not necessarily lead to better results if the prices still have a wide range, so we decided to try and limit the dataset to just one car, the most common car in our dataset, which was the **VW Golf**. This way we hoped to get rid of the outliers and achieve better results.

Here we decided to approach the process in two ways, taking all VW Golfs, including for example the performance models, such as the GTI and R, and then taking only the regular models. The following table shows the results of these two separate approaches.

Dataset	MAE	MAPE	MSE	R^2
All VW Golfs (Train)	2996.59	22.97%	38246625.51	0.9951
All VW Golfs (Test)	9015.96	36.45%	1174102816.20	0.8768
Base VW Golfs (Train)	563.78	8.60%	989554.54	0.9913
Base VW Golfs (Test)	2035.60	25.32%	13413777.77	0.8936

Table 7: XGBoost Results on VW Golf Dataset

We can see that the results are much better than on the whole dataset and the narrower the dataset the better the results are. The 25% MAPE and MAE of 2k euros is much more acceptable for the Golf, compared to the overall 30k MAE we had before. The predicted vs actual plots also look much nicer.

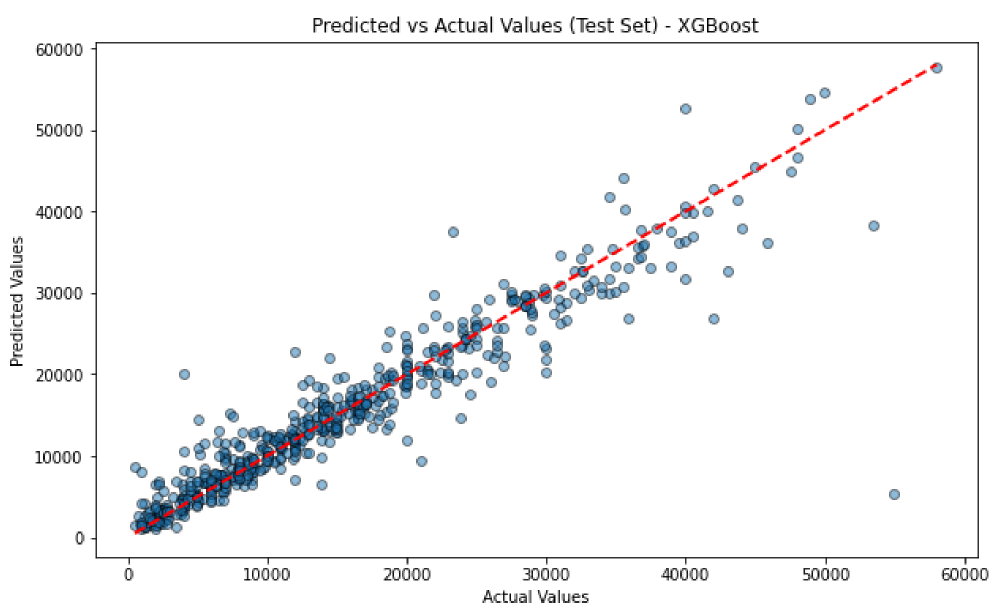


Figure 22: XGBoost Predicted vs Actual Prices on Base VW Golf Dataset. Source: Own work

This plot does not show the common problem of overestimating cheaper cars we have seen from more general models. Probably due to the fact that there aren't premium models in the dataset, so the model does not have to learn to predict these outliers.

4.2.3 Cleaning Prices More Harshly

Another possible approach we wanted to try was to clean the all of our numerical features more harhsly as many of them were very skewed. In the end we used a mixture of IQR and strict numerical capping in conjunction with a by car_title based IQR outlier removal which I have explained before in section 3.2. Training the XGBoost model, we had an idea that this dataset would perform much better than the base cleaned dataset, but we did not expect it to even outperform the VW Golf dataset, which it has done by quite a large margin.

Metric	Train	Test
Mean Absolute Error (MAE)	500.7276	1806.8305
Mean Squared Error (MSE)	522615.9210	10388687.9100
R-squared (R^2)	0.9972	0.9439
Mean Absolute Percentage Error (MAPE)	6.44%	16.63%

Table 8: XGBoost Results on Harshly Cleaned Dataset

These results are very impressive, with a MAE of just *1806.8305* EUR, which is actually almost usable on a price range of *500* EUR to *64k* EUR. Even the R^2 of *0.9439* is very much comparable with that of the scientific literature. Our MAPE is also down to *16.63%*, which is a very acceptable value. In this case, we found our best configuration via hyperparameter tuning and making the model more complex seems to have just worsened the results. Being satisfied with these results, we tried inspecting the residuals and the predicted vs actual prices, which look very nice, with the model not overestimating the cheaper cars and not underestimating the premium ones either by a large margin as we have seen before.

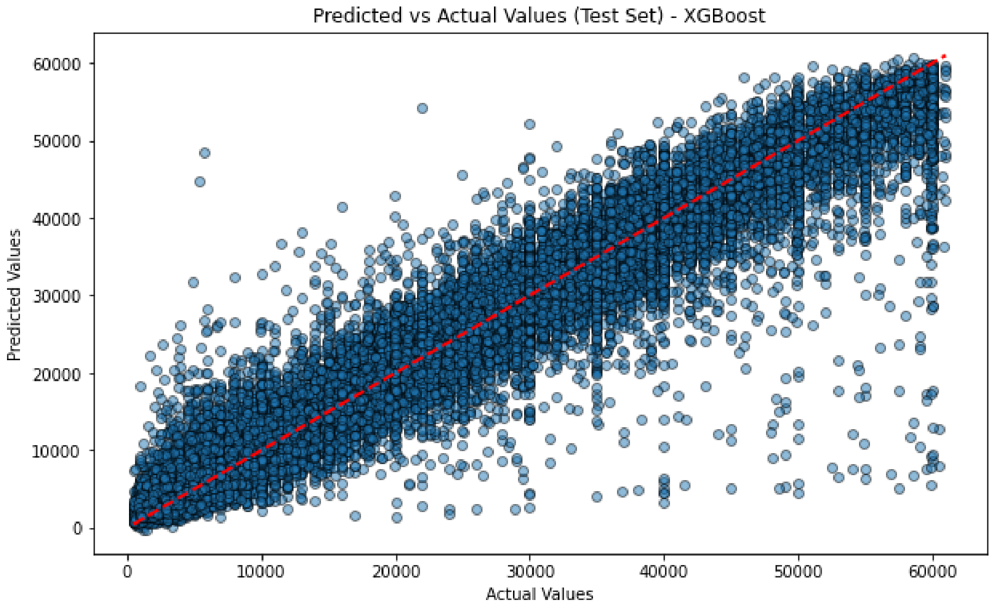


Figure 23: Predicted vs Actual - Harshly Cleaned. Source: Own work

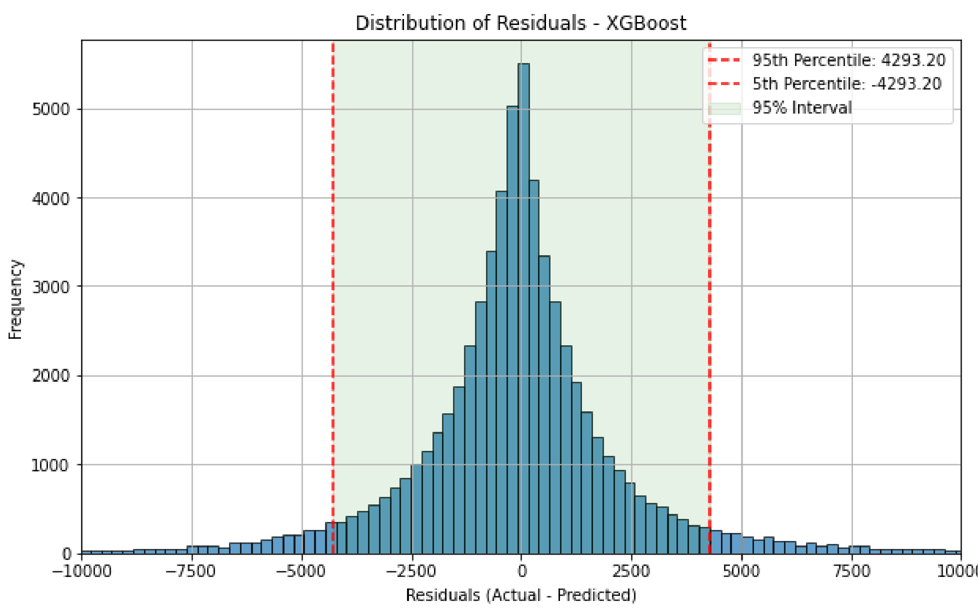


Figure 24: Residuals Distribution - Harshly Cleaned. Source: Own work

One more plot we could inspect is the feature importance, which show a completely realistic picture of what may influence the price of a car. In this more grounded dataset, we see **used_type** as the most important factor, which is very reasonable. Aside the classic features such as power and mileage many lifelike features make an appearance, such as the automatic gearbox, whether the car is a pickup or if it has 4WD.

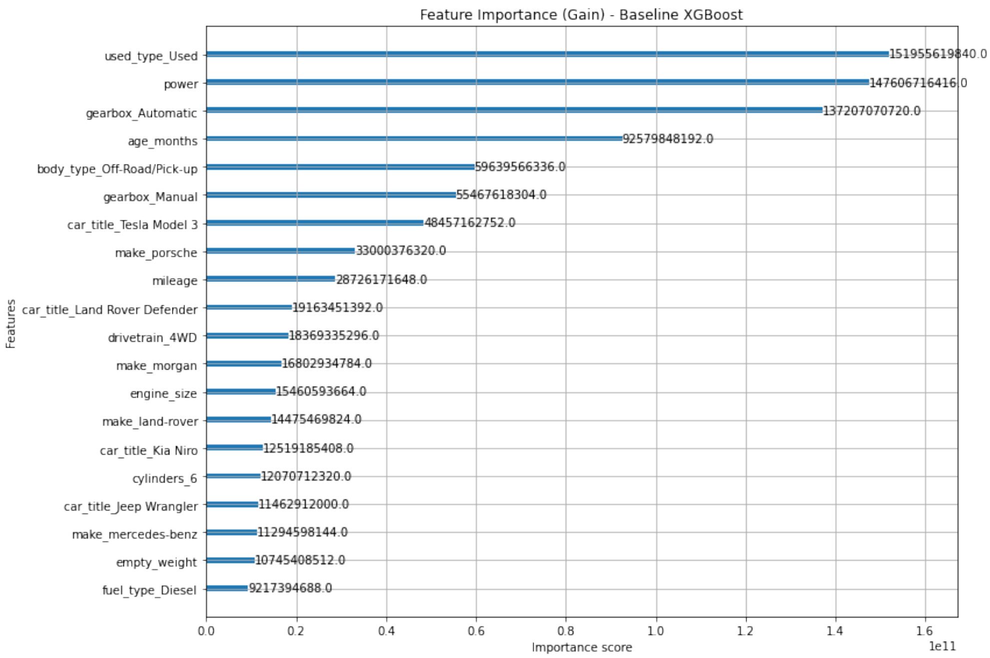


Figure 25: Feature Importance - Harshly Cleaned. Source: Own work

4.2.4 Adding Extra Features

Having seen how well the harshly cleaned dataset performed, we decided to try and add the extra leftover features to this dataset, to see if we could get an even better result. For this we tried using our composite **;** delimitted features, as explained in section 3.1.2. We did a short analysis and found that we would generally keep those additional features that were present for at least *5%* of the dataset, plus we removed those features that were too common in the modern age, e.g. "Radio" or "ABS". In the end, we were left with *90* additional features, which we then one-hot encoded and added to our dataset.

As usual we trained the XGBoost based on the harshly cleaned dataset, but this time with the additional features we got marginally better results.

Metric	Train	Test
Mean Absolute Error (MAE)	1212.6710	1735.6964
Mean Squared Error (MSE)	3041479.5255	9336662.8078
R-squared (R^2)	0.9835	0.9496
Mean Absolute Percentage Error (MAPE)	12.79%	16.344%

Table 9: XGBoost Results on Harshly Cleaned Dataset with Extra Features

This is a modest *71* EUR improvement on the MAE, *0.57%* improvement on R^2 and almost *0.3%* improvement on MAPE, which is not too bad, but not too impressive either.

We can also see that for this best model we have found a configuration that did not fit the training data as much as the previous one, given more boosting rounds, we probably could have achieved that coveted *0.95* R^2 on the test set, but this was already using *10000* boosting rounds, taking a long time.

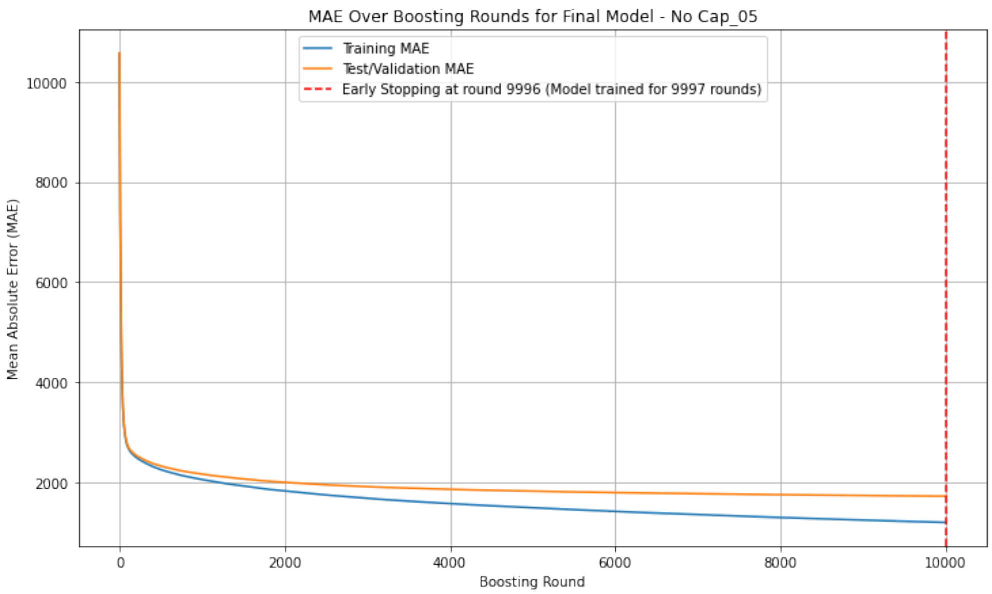


Figure 26: MAE Loss Curve of the Harshly Cleaned Extra Features Model. Source: Own work

The top 20 feature importance of this model shows that these additional features are generally not as important as our base numerical features and they are less important than our make and car_title features, but some of them can still be significant predictors, e.g. **electric_tailgate** which is a very good indicator of a premium trim level, usually present in luxury cars.

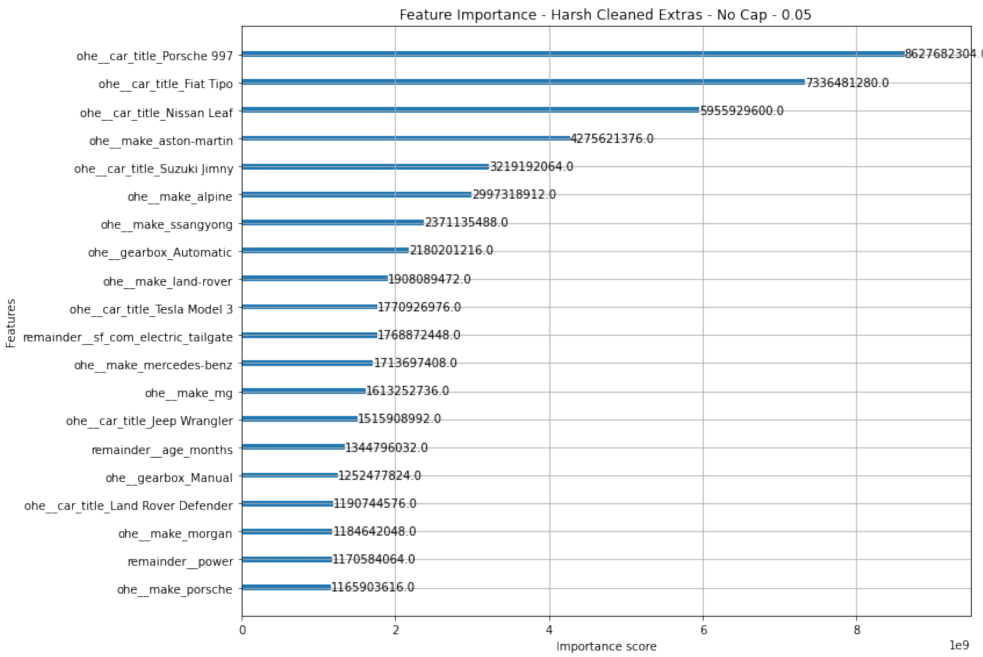


Figure 27: Feature Importance - Harshly Cleaned with Extra Features. Source: Own work

As for the predicted vs actual prices and residuals distribution plot, they are visibly better compared to even the harshly cleaned dataset. The residuals distribution was plotted on the same scale as the previous one, thus they are directly comparable, and we can see that the distribution is not as sharp, but it is more balanced, with more of the predictions being closer to the actual prices.

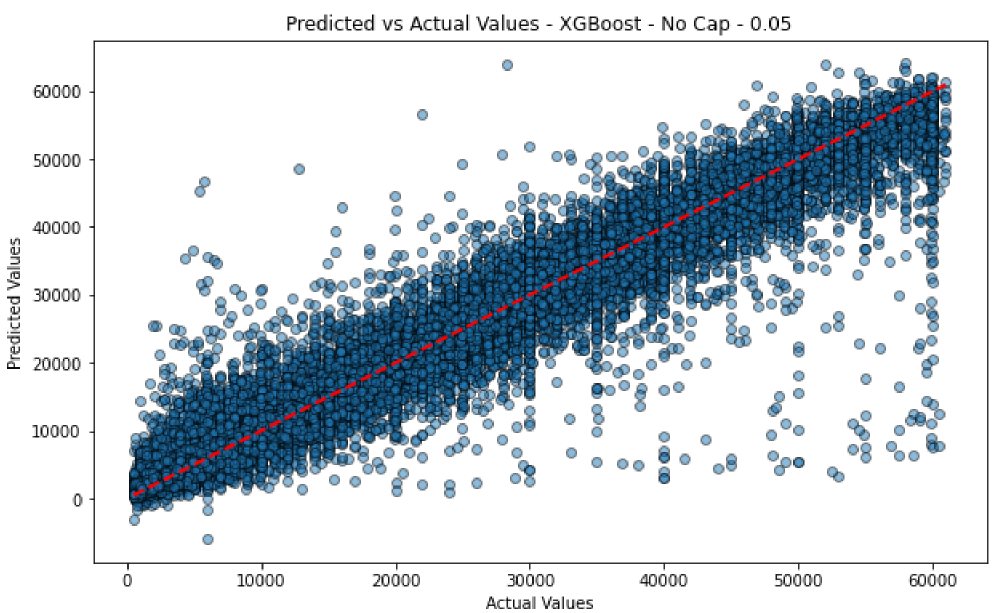


Figure 28: Predicted vs Actual - Harshly Cleaned with Extra Features. Source: Own work

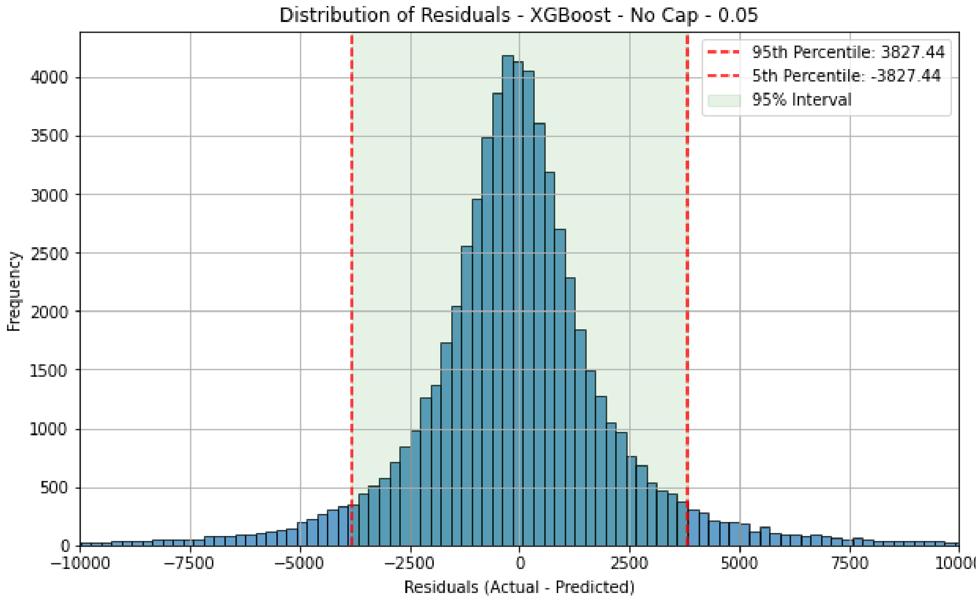


Figure 29: Residuals Distribution - Harshly Cleaned with Extra Features. Source: Own work

5 Discussion

Interpret and analyze the results, and discuss possible future work and improvements.

TODO: FINISH DISCUSSION

6 Conclusion

Provide a summary of the project, key findings, and recommendations. (Approximately 200–350 words.)

TODO: FINISH CONCLUSION

7 Bibliography

[1] E. Gegic and B. Isakovic, "Car Price Prediction using Machine Learning Techniques," *TEM Journal*, vol. 8, no. 1, pp. 113–118, 2019, ISSN: 2217-8309, 2217-8333. [Online]. Available: `https://www.ceeol.com/search/article-detail?id=746689` (visited on 05/20/2025).

[2] L. Bukvić, J. Pašagić Škrinjar, T. Fratrović, and B. Abramović, “Price Prediction and Classification of Used-Vehicles Using Supervised Machine Learning,” *Sustainability*, vol. 14, no. 24, p. 17 034, 24 Jan. 2022, ISSN: 2071-1050. DOI: 10.3390/su142417034. [Online]. Available: https://www.mdpi.com/2071-1050/14/24/17034 (visited on 05/20/2025).

[3] C. Jin, “Price Prediction of Used Cars Using Machine Learning,” in *2021 IEEE International Conference on Emergency Science and Information Technology (ICESIT)*, Nov. 2021, pp. 223–230. DOI: 10.1109/ICESIT53460.2021.9696839. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9696839 (visited on 05/20/2025).

[4] Y. Zheng, “Machine Learning Optimization and Challenges in Used Car Price Prediction,” *ITM Web of Conferences*, vol. 70, p. 04 032, 2025, ISSN: 2271-2097. DOI: 10.1051/itmconf/20257004032. [Online]. Available: https://www.itm-conferences.org/articles/itmconf/abs/2025/01/itmconf_dai2024_04032/itmconf_dai2024_04032.html (visited on 05/20/2025).

[5] M. Nandan and D. Ghosh, “Pre-owned car price prediction by employing machine learning techniques,” *Journal of Decision Analytics and Intelligent Computing*, vol. 3, no. 1, pp. 167–184, 1 Oct. 8, 2023, ISSN: 2787-2572. DOI: 10.31181/jdaic10008102023n. [Online]. Available: https://www.jdaic-journal.org/index.php/about/article/view/31 (visited on 05/21/2025).

[6] S. Voß and S. Lessmann, “Resale Price Prediction in the Used Car Market,”

[7] E. Liu, J. Li, A. Zheng, H. Liu, and T. Jiang, “Research on the Prediction Model of the Used Car Price in View of the PSO-GRA-BP Neural Network,” *Sustainability*, vol. 14, no. 15, p. 8993, 15 Jan. 2022, ISSN: 2071-1050. DOI: 10.3390/su14158993. [Online]. Available: https://www.mdpi.com/2071-1050/14/15/8993 (visited on 05/20/2025).

[8] S. Shaprapawad, P. Borugadda, and N. Koshika, “Car Price Prediction:An Application of Machine Learning,” in *2023 International Conference on Inventive Computation Technologies (ICICT)*, Apr. 2023, pp. 242–248. DOI: 10.1109/ICICT57646.2023.10134142. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10134142 (visited on 05/23/2025).

[9] Y. Li, Y. Li, and Y. Liu, “Research on used car price prediction based on random forest and LightGBM,” in *2022 IEEE 2nd International Conference on Data Science and Computer Application (ICDSCA)*, Oct. 2022, pp. 539–543. DOI: 10.1109/ICDSCA56264.2022.9988116. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9988116 (visited on 05/20/2025).

[10] C. Chen, L. Hao, and C. Xu, “Comparative analysis of used car price evaluation models,” *AIP Conference Proceedings*, vol. 1839, no. 1, p. 020 165, May 8, 2017, ISSN: 0094-243X. DOI: 10.1063/1.4982530. [Online]. Available: https://doi.org/10.1063/1.4982530 (visited on 05/20/2025).

[11] J. Huang, Z. Yu, Z. Ning, and D. Hu, “Used Car Price Prediction Analysis Based on Machine Learning,” presented at the 2022 International Conference on Artificial Intelligence, Internet and Digital Economy (ICAID 2022), Atlantis Press, Dec. 2, 2022, pp. 356–364, ISBN: 978-94-6463-010-7. DOI: 10.2991/978-94-6463-010-7_37. [Online]. Available: https://www.atlantis-press.com/proceedings/icaid-22/125977128 (visited on 05/20/2025).